CHAPTER 9

Server-Side and PHP Slim

Building RESTful APIs as the backbone of cross-platform applications

SCSM2223 — Cross-Platform Application Development
Faculty of Computing • Universiti Teknologi Malaysia
Assoc. Prof. Dr Hishammuddin bin Asmuni

AT THE END OF THIS CHAPTER

Learning Objectives

C1 Define the role of the server-side and explain RESTful API design principles.

C2 Differentiate HTTP methods and standard status codes used in REST APIs.

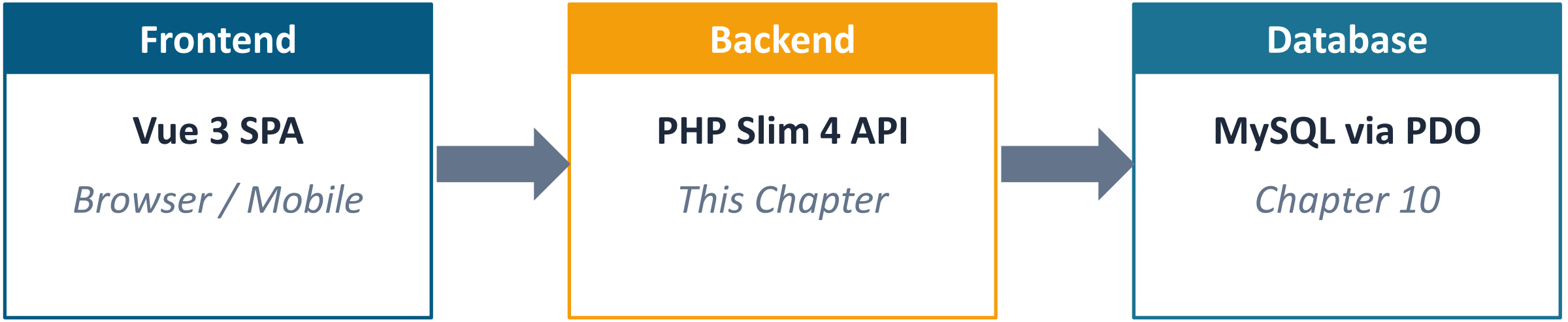
C2 Describe the architecture and request lifecycle of the PHP Slim 4 framework.

C3 Set up a Slim project with Composer on Laragon and define basic routes.

C3 Read JSON request bodies, return JSON responses, and use route parameters.

C3 Apply middleware for cross-cutting concerns (CORS, logging, JSON parsing).

Where the Server-Side Sits



HTTP requests with JSON bodies travel left → right; JSON responses travel right → left.

In this chapter we focus on the BACKEND: receiving requests, executing logic, and returning JSON.

PROCESS FLOW – WEB APPLICATION (VUE 3 + VITE, PHP SLIM 4, FILE-BASED JSON STORAGE)

1 WEB BROWSER



User opens application in browser

1 User opens application in browser

2 FRONTEND (VUE 3 + VITE)



Vue Application

- Sends HTTP requests
- Manages state (Pinia)
- Renders UI
- Displays book list

2 Frontend sends HTTP request to REST API
e.g. GET /api/books

7 API returns JSON response to frontend

8 Frontend updates state and renders book list

3 BACKEND API (PHP SLIM 4)



Slim 4 REST API

- Routes handle requests
- Controllers validate & orchestrate
- BookStorage service manages in-memory books array
- Returns JSON responses

3 API calls BookStorage->load()

4 load() checks var/books.json

5 If exists: read JSON data into memory

6 If not exists: seed from Data/books.php, then create var/books.json

4 FILE STORAGE + SEED DATA

A. SEED SOURCE
Data/books.php



Original seed array of books

B. PERSISTED FILE
var/books.json



Current working data (mutations persisted)

During runtime, BookStorage keeps the books in a PHP array in memory after load().

FRONTEND DETAILS (Vue 3 + Vite)

Project Structure (simplified)

```
src/
  assets/
  components/
    BookList.vue
    BookItem.vue
  views/
    BooksView.vue
  stores/
    bookStore.js (Pinia)
  services/
    api.js (axios instance)
  App.vue
  main.js
```

Component Interaction Flow

App.vue → BooksView.vue → useBookStore (Pinia) → api.js (axios) → REST API (PHP Slim 4)

JSON Response

UI State Flow

User Action (e.g. click, add, edit, delete) → View mounted → store.fetchBooks() or other action → API Call → State Update (Pinia Store) → UI re-render

BACKEND DETAILS (Slim 4)

Request Flow

HTTP Request (GET /api/books) → Slim Router → BooksController → BookStorage Service (load() / save() / reset()) → JSON Response

Key Files (simplified)

- app/Controllers/BooksController.php
- app/Services/BookStorage.php
- Data/books.php
- var/books.json
- routes.php
- dependencies.php

Example Routes

GET	/api/books
GET	/api/books/{id}
POST	/api/books
PUT	/api/books/{id}
DELETE	/api/books/{id}
POST	/api/reset

BookStorage Service (simplified pseudocode)

```
load():
if file exists at var/books.json:
    $json = file_get_contents('var/books.json');
    $books = json_decode($json, true);
    return $books;
else:
    $books = require __DIR__.'/../../Data/books.php';
    $this->save($books);
    return $books;
```

```
save($books):
$json = json_encode($books,
  JSON_PRETTY_PRINT);

file_put_contents(
  'var/books.json', $json
);

return $books;
```

```
reset():
$books = require
  _DIR__.'/../../Data/books.php';

$this->save($books);

return $books;
```

FILE STORAGE DETAILS

A. SEED SOURCE: Data/books.php

Original seed array of books

```
return [
  {
    "id" => 1,
    "title" => "Clean Code",
    "author" => "Robert C. Martin",
    "isbn" => "9780132350884",
    "published_at" => "2008-08-00"
  },
  ...
];
```

Book Object Schema (fields)

id (int)	title (string)	author (string)	isbn (string)	published_at (YYYY-MM-DD)

EXAMPLE HTTP EXCHANGE

GET /api/books — Response: 200 OK

Request

```
GET /api/books
Host: localhost:8000
Accept: application/json
```

```
[
  {
    "id": 1,
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "isbn": "9780132350884",
    "published_at": "2008-08-00"
  },
  ...
]
```

POST /api/reset — Response: 200 OK

Request

```
POST /api/reset
Host: localhost:8000
Accept: application/json
```

```
[
  {
    "id": 1,
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "isbn": "9780132350884",
    "published_at": "2008-08-00"
  },
  ...
]
```

END-TO-END FLOW SUMMARY

1. Browser loads Vue app.
2. Vue calls Slim 4 REST API.
3. Slim uses BookStorage->load() to populate an in-memory array from var/books.json, or seeds it from Data/books.php if missing.
4. CRUD operations update the in-memory array and call save() to persist changes to var/books.json.
5. POST /api/reset calls reset() to restore the original seed data.
6. API returns JSON and the frontend re-renders.

All data is stored in var/books.json. No database required!

WHAT THE SERVER DOES

Role of the Server-Side

1 Receive Requests

Listens on HTTP, parses URL, headers, query, and body.

2 Apply Business Logic

Validates input, enforces rules, computes results.

3 Talk to Data

Reads/writes to database, file system, or other APIs.

4 Authenticate & Authorize

Verifies who the user is and what they can do.

5 Return a Response

Sends back JSON, status code, and useful headers.

6 Cross-Cutting Concerns

Logging, error handling, CORS, rate limiting.

ARCHITECTURAL STYLE

What is REST?

REpresentational State Transfer

Coined by Roy Fielding (2000) — the dominant style for web APIs today.

Core idea

Everything is a **resource** (a noun, not an action).

Each resource has a stable URI such as `/products/12`.

Clients use standard HTTP verbs to manipulate it: GET, POST, PUT, DELETE.

The server returns a representation of the resource — typically JSON.

Key constraints

Stateless
Every request carries everything the server needs.

Uniform interface
Same set of HTTP verbs across every resource.

Client–server separation
Frontend and backend evolve independently.

Cacheable
Responses indicate whether they may be cached.

HTTP Methods Used in REST

VERB	USE	PROPERTIES	EXAMPLE
GET	Retrieve a resource	*Safe • Idempotent*	`GET /products`
POST	Create a new resource	*Not safe • Not idempotent*	`POST /products`
PUT	Replace a resource	*Idempotent*	`PUT /products/12`
PATCH	Partial update	*Not idempotent (typically)*	`PATCH /products/12`
DELETE	Remove a resource	*Idempotent*	`DELETE /products/12`

Common HTTP Status Codes

2xx	**Success**	
200	**OK**	GET / PUT / DELETE succeeded
201	**Created**	POST created a new resource
204	**No Content**	Success, but no body to return
4xx	**Client error**	
400	**Bad Request**	Validation failed / malformed JSON
401	**Unauthorized**	Missing / invalid token
403	**Forbidden**	Authenticated but not allowed
404	**Not Found**	Resource doesn't exist
5xx	**Server error**	
500	**Internal Server Error**	Unhandled exception in server code
503	**Service Unavailable**	Backend temporarily down

RESTful URI Patterns

Use plural nouns. Express hierarchy with nesting. Avoid verbs in URIs.

GOOD		BAD	
GET	/products	GET	/getAllProducts
GET	/products/12	GET	/productById?id=12
POST	/products	POST	/createProduct
PUT	/products/12	POST	/updateProduct/12
DELETE	/products/12	GET	/deleteProduct/12
GET	/vendors/3/menu-items	GET	/listMenuFor?vendor=3

Anatomy of an HTTP Request & Response

REQUEST →

```
POST /api/products HTTP/1.1
Host: localhost:8000
Authorization: Bearer
eyJhbGc...
Content-Type: application/json

{
  "name":  "Notebook",
  "price": 12.50
}
```

← RESPONSE

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/products/15

{
  "id":      15,
  "name":    "Notebook",
  "price":   12.50,
  "message": "Product created"
}
```

Three things to inspect on every call: STATUS CODE, HEADERS, and BODY.

JSON — JavaScript Object Notation

Why JSON?

- Lightweight — text only, no schema overhead.
- Language independent — parsed by every language.
- Maps cleanly to JS objects, PHP assoc arrays, and Vue reactive state.
- Native browser support via JSON.parse / JSON.stringify.
- Standard Content-Type: application/json.

```
{
  "id":        12,
  "name":      "Hashim",
  "isActive": true,
  "roles":     ["admin",
"editor"],
  "address":  {
    "city":   "Skudai",
    "zip":    "81310"
  }
}
```

What is the PHP Slim Framework?

"Slim is a PHP micro framework that helps you quickly write simple yet powerful web applications and APIs."
— slimframework.com

1

Micro

Tiny core, ~few KB. Bring only what you need.

2

PSR-7 / PSR-15

Standard request/response and middleware contracts.

3

Routing-first

Powerful route patterns with parameters and groups.

4

Plays well with others

Add Eloquent, Twig, JWT — your choice.

Slim vs Alternatives

ASPECT	Raw PHP	Slim 4 (this course)	Laravel / Symfony
Footprint	Tiny	Small (a few MB)	Large (full framework)
Routing	Manual switch on URL	Built-in router	Built-in router
Middleware	Custom	PSR-15 standard	Pipeline
Learning curve	Low entry, scales badly	Low entry, scales well	Steep but feature-rich
Best for	Tiny scripts	REST APIs / micro-services	Full web apps + admin panels

Slim hits the sweet spot for our course: REST API focus, manageable scope for a 4-person team in 7 weeks.

How a Slim Request Flows

1. Browser sends request
2. Web server (Apache/Nginx) hits public/index.php
3. Slim app parses request → PSR-7 object
4. Middleware stack runs (auth, CORS, logging…)
5. Router dispatches to matching handler
6. Handler builds PSR-7 response
7. Middleware unwraps in reverse
8. Web server returns response to browser

Mental model:

`Request → M1 → M2 → Route Handler → Response ← M2 ← M1`

Middleware wraps the handler like an onion — each layer can read the request, mutate the response, or short-circuit (e.g., reject unauthenticated calls).

Installing Slim with Composer

Use Laragon's built-in Composer (or install via getcomposer.org). All commands are run from the project root.

```
# 1. Create the project folder inside laragon\www
cd C:\laragon\www
mkdir books-api && cd books-api

# 2. Initialise Composer
composer init --name="utm/books-api" --no-interaction

# 3. Install Slim 4 + the official PSR-7 implementation
composer require slim/slim:"4.*" slim/psr7

# 4. (Optional but recommended) PHP-DI for dependency injection
composer require php-di/php-di

# 5. Verify
composer show | findstr slim
```

Recommended Slim Project Structure

```
books-api/
├── public/
│   ├── index.php          ← entry point (Apache/Nginx points here)
│   └── .htaccess          ← rewrite rules for clean URLs
├── src/
│   ├── routes.php         ← all route definitions
│   ├── Controllers/
│   │   └── BookController.php
│   ├── Middleware/
│   │   └── JsonBodyParser.php
│   └── Data/
│       └── books.php      ← seed data (Chapter 9, no DB yet)
├── vendor/                ← installed by Composer (don't edit)
├── composer.json
└── README.md
```

Hello World in Slim 4

```php
<?php  // public/index.php
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;
use Slim\Factory\AppFactory;

require __DIR__ . '/../vendor/autoload.php';

$app = AppFactory::create();
$app->addRoutingMiddleware();
$app->addErrorMiddleware(true, true, true);

$app->get('/', function (Request $req, Response $res) {
    $res->getBody()->write('Hello, Slim!');
    return $res;
});

$app->run();
```

Reading the code

`AppFactory::create()`
Builds the Slim app.

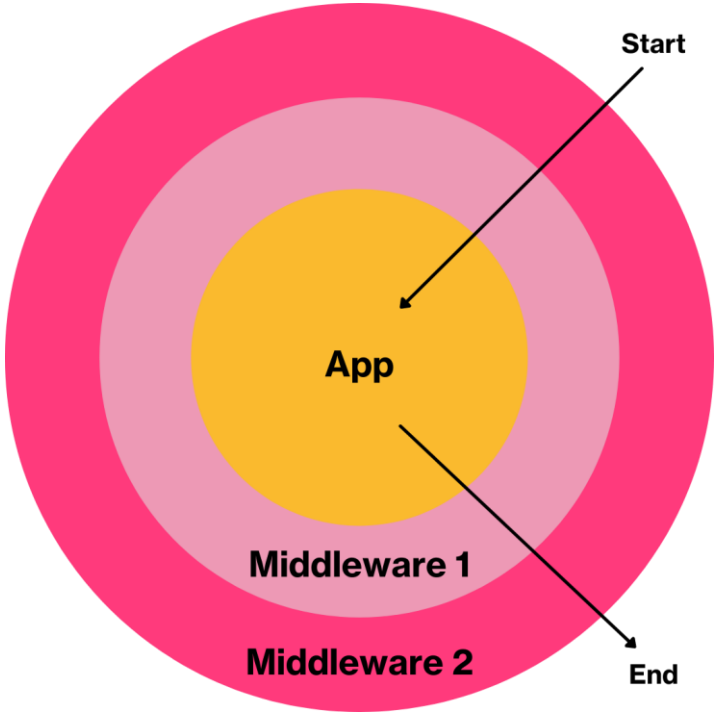
`addRoutingMiddleware()`
Enables route dispatch.

`addErrorMiddleware(...)`
Catches exceptions, returns proper errors.

`$app->get('/', ...)`
Defines a GET route at the root path.

`$app->run()`
Starts the request lifecycle.

In a Slim application, each route invokes a callback function that returns an HTTP response object. The code above begins by bringing in all the PSR-7 components that the app needs. It then instantiates the app and attaches two middleware classes to it.
The Slim middleware architecture is derived from the PSR-15 middleware interface. This is a series of concentric layers that encircle the core application, such that the last middleware added will be the first to be executed.



This concentric architecture means that when the Slim application is started, the request object passes through a series of middleware classes, or layers, working from the outside inward. The request enters the outermost middleware, then moves on to the next outermost middleware, continuing until it reaches the application.
For this reason, to catch any errors in the application, you call `$app->addErrorMiddleware(true, true, true)` last, otherwise any errors thrown from the other middlewares will not be caught.
Once the request object reaches the Slim application, the relevant route is dispatched, resulting in a response object exiting the concentric layers from the inside out. When the object passes the outermost middleware, it is turned into a raw HTTP response and then returned to the client.

Route Patterns You'll Use Daily

```
// Static path
$app->get('/about', fn(Request $r, Response $s) => $s->getBody()->write('about') ?: $s);

// Path parameter (id is captured)
$app->get('/books/{id}', function (Request $r, Response $s, array $args) {
    $id = (int)$args['id'];
    $s->getBody()->write(json_encode(['id' => $id]));
    return $s->withHeader('Content-Type', 'application/json');
});

// Multiple verbs on the same path
$app->post('/books',          BookController::class . ':create');
$app->put('/books/{id}',      BookController::class . ':update');
$app->delete('/books/{id}',   BookController::class . ':delete');

// Route group with shared prefix
$app->group('/api', function ($g) {
    $g->get('/books',         BookController::class . ':index');
    $g->get('/books/{id}',    BookController::class . ':show');
});
```

Reading Inputs from the Request

```
// 1. Path parameters — third callback argument
$app->get('/books/{id}', function (Request $r, Response $s, array $args) {
    $id = (int)$args['id'];   // /books/12 → 12
    return $s;
});

// 2. Query string — ?q=harry&limit=10
$app->get('/books', function (Request $r, Response $s) {
    $params = $r->getQueryParams();
    $q      = $params['q']     ?? '';
    $limit  = (int)($params['limit'] ?? 20);
});

// 3. JSON body for POST/PUT
$app->post('/books', function (Request $r, Response $s) {
    $data = json_decode((string)$r->getBody(), true);  // assoc array
    $title = $data['title'] ?? null;
});
```

Building a JSON Response

```
// Convention: always return JSON + correct status code.
function jsonResponse(Response $res, $data, int $status = 200): Response {
    $res->getBody()->write(json_encode($data));
    return $res
        ->withHeader('Content-Type', 'application/json')
        ->withStatus($status);
}

$app->post('/books', function (Request $r, Response $s) {
    $data = json_decode((string)$r->getBody(), true);
    if (empty($data['title'])) {
        return jsonResponse($s, ['error' => 'title is required'], 400);
    }
    $book = ['id' => 99, 'title' => $data['title']];
    return jsonResponse($s, $book, 201)
        ->withHeader('Location', '/api/books/' . $book['id']);
});

// Tip: Slim's PSR-7 response object is IMMUTABLE.
// withHeader / withStatus return a NEW object — always return it.
```

Middleware: Cross-Cutting Logic

Common uses

- JSON body parsing
- CORS headers
- Authentication (JWT)
- Logging / timing
- Rate limiting
- Error formatting

```
// Custom middleware — parses JSON bodies into the request
$app->add(function (Request $req, $handler) {
    if (str_starts_with($req->getHeaderLine('Content-Type'),
'application/json')) {
        $body = (string)$req->getBody();
        $req = $req->withParsedBody(json_decode($body, true) ?? []);
    }
    return $handler->handle($req);  // pass on to next layer
});

// CORS — let a Vue dev server on :5173 call our API
$app->add(function (Request $req, $handler) {
    $res = $handler->handle($req);
    return $res
        ->withHeader('Access-Control-Allow-Origin', '*')
        ->withHeader('Access-Control-Allow-Headers', 'Content-Type,
Authorization')
        ->withHeader('Access-Control-Allow-Methods', 'GET, POST, PUT,
DELETE, OPTIONS');
});
```

Worked Example — BookController

```php
<?php  // src/Controllers/BookController.php
namespace App\Controllers;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;

class BookController {
  private static array $books = [
    ['id'=>1,'title'=>'Clean Code',         'author'=>'Robert C. Martin'],
    ['id'=>2,'title'=>'Eloquent JavaScript','author'=>'Marijn Haverbeke'],
  ];
  public function index(Request $r, Response $s): Response {
    return $this->json($s, self::$books);
  }
  public function show(Request $r, Response $s, array $args): Response {
    $id = (int)$args['id'];
    $found = array_values(array_filter(self::$books, fn($b)=>$b['id']===$id))[0] ?? null;
    return $found ? $this->json($s, $found)
                  : $this->json($s, ['error'=>'not found'], 404);
  }
  private function json(Response $s, $d, int $code=200): Response {
    $s->getBody()->write(json_encode($d));
    return $s->withHeader('Content-Type','application/json')->withStatus($code);
  }
}
```

WATCH OUT

Common Pitfalls & Best Practices

✗ **Forgetting to RETURN the response**
withHeader/withStatus are immutable — always return the new object.

✗ **Wrong Content-Type**
Use application/json on every JSON response.

✗ **Stringifying twice**
json_encode($data) once; don't wrap with another json_encode.

✓ **Always validate inputs**
Treat anything from the client as hostile until proven otherwise.

✓ **Use route groups**
Group /api routes once, apply auth middleware to the group.

✓ **Return correct status codes**
201 on create, 204 on delete, 400 / 401 / 404 / 500 where appropriate.

Summary — Chapter 9

Server-side	Validates input, enforces logic, talks to data, returns JSON.
REST	Resources are nouns; HTTP verbs act on them; statelessness wins.
HTTP	Know the verbs (GET/POST/PUT/PATCH/DELETE) and the codes (2xx/4xx/5xx).
Slim 4	Micro framework on PSR-7/15; routes + handlers + middleware.
Practice	You can scaffold, define routes, parse JSON, and return responses.

Next Up

Lab Hands-On — Build a Books REST API

Five GET / POST / PUT / DELETE endpoints in PHP Slim 4 — running on Laragon, tested with curl/Postman.

- Set up Laragon + Composer
- Scaffold Slim 4 project
- Define five RESTful routes
- Test each with Postman / curl
- Bonus: add a custom middleware